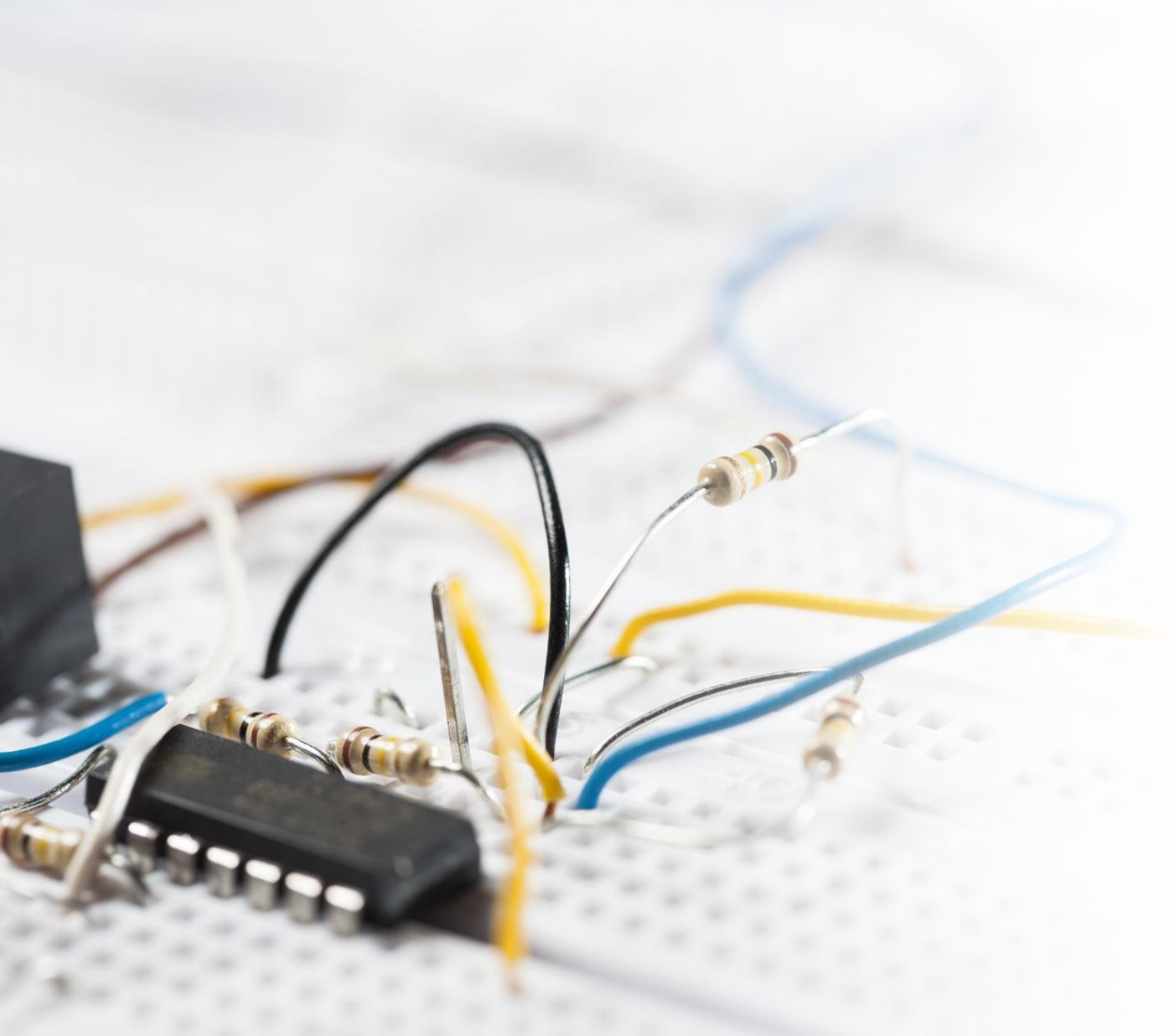




Projet arduino main robotique

PRÉSENTATION

Introduction :

Aujourd'hui je vais vous présenter mon projet : la réalisation d'une main robotique biomimétique commandée par les mouvements d'une main humaine.

Le but du projet est de reproduire mécaniquement les mouvements naturels des doigts en temps réel grâce à des capteurs et des moteurs.

Ce projet s'inspire du fonctionnement des prothèses modernes et de la robotique humanoïde.

Problématique

La question principale est :

Comment transformer un mouvement humain en mouvement mécanique ?

Pour cela il faut :

mesurer la position d'un doigt

traiter l'information

commander un moteur proportionnellement

Principe de fonctionnement

Le fonctionnement est basé sur une chaîne d'information et d'énergie.

Quand je plie mon doigt :

Le capteur flex se plie

Sa résistance change

Arduino lit la valeur

Le programme calcule un angle

Le servomoteur reproduit le mouvement

Donc chaque doigt humain pilote directement un doigt robotique.

Chaîne d'information

Le système comporte plusieurs étapes :

Acquisition :

Les capteurs mesurent la flexion

Conversion :

Arduino convertit la tension en nombre grâce à un convertisseur analogique-numérique sur 10 bits (0 à 1023)

Traitement :

Le programme calcule l'angle du doigt

Action :

Le moteur reçoit un signal PWM et se positionne

Le capteur flex

Le capteur flex est une résistance variable.

Doigt droit → résistance faible

Doigt plié → résistance élevée

Donc la tension change selon la flexion.

Mais Arduino ne peut pas mesurer une résistance directement, il doit mesurer une tension.

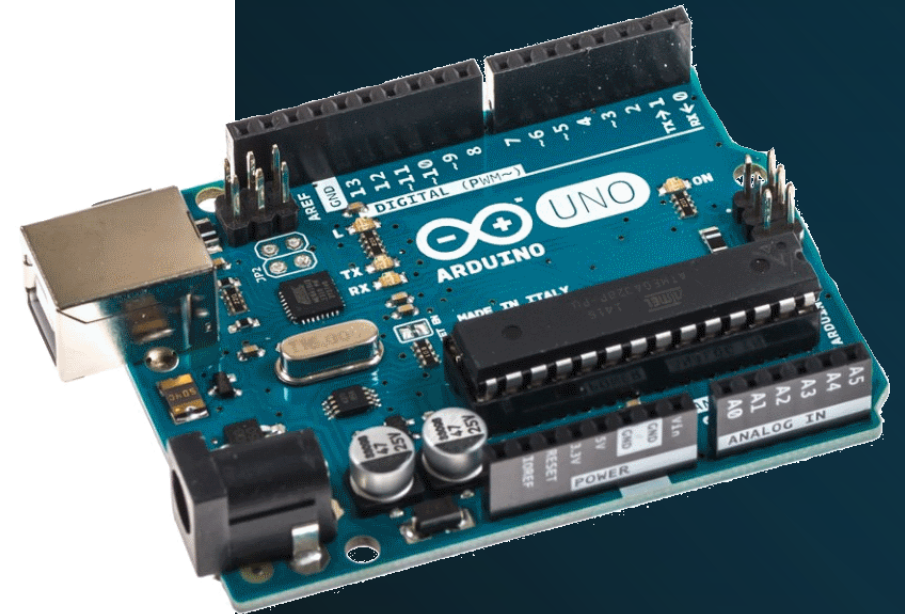


Le pont diviseur de tension

- Pour transformer la résistance en tension on utilise un pont diviseur.
- Montage :
- 5V → Flex → Arduino → Résistance → GND
- Ce montage permet d'obtenir une tension variable entre 0V et 5V.
- Sans ce montage la mesure serait instable ou toujours 0/1023.

La carte Arduino UNO

- Le cerveau du système est un microcontrôleur ATmega328P.
- Ses rôles :
 - lire les entrées analogiques
 - effectuer les calculs
 - générer les signaux moteurs
- Il possède :
 - 6 entrées analogiques
 - des sorties PWM pour moteurs





Les servomoteurs

Les actionneurs sont des servomoteurs.

Caractéristiques :

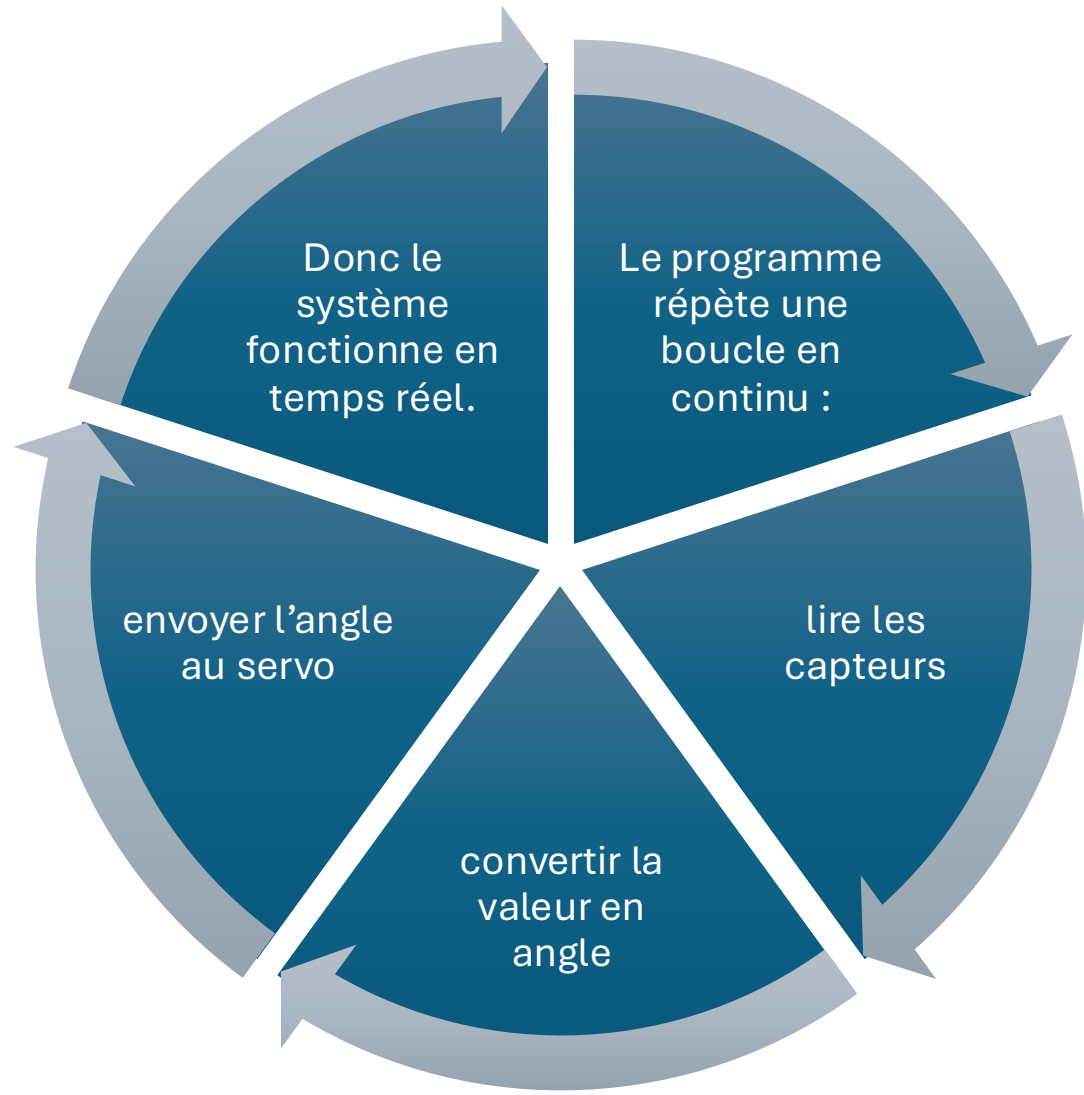
rotation contrôlée de 0° à 180°

position précise

commande par signal PWM toutes les 20 ms

Chaque servo correspond à un doigt.

Fonctionnement du programme



Donc le système fonctionne en temps réel.

Image du code
arduino:

```
// C++ code
//
#include <Servo.h>
Servo servo1;
Servo servo2;
Servo servo3;
Servo servo4;
Servo servo5;
void setup()
{
  servo1.attach (5);
  servo2.attach (6);
  servo3.attach (9);
  servo4.attach (10);
  servo5.attach (11);
  Serial.begin (9600);
}
void loop()
{
  int flex_sensor1 = analogRead (A4);
  int flex_sensor2 = analogRead (A3); //flex_sensor()=();
  int flex_sensor3 = analogRead (A2);
  int flex_sensor4 = analogRead (A1);
  int flex_sensor5 = analogRead (A0);

  flex_sensor1 = map (flex_sensor1, 863, 510, 0, 180);
  flex_sensor2 = map (flex_sensor2, 863, 510, 0, 180);
  flex_sensor3 = map (flex_sensor3, 863, 510, 0, 180);
  flex_sensor4 = map (flex_sensor4, 863, 510, 0, 180);
  flex_sensor5 = map (flex_sensor5, 863, 510, 0, 180);

  servo1.write (flex_sensor1);
  servo2.write (flex_sensor2);
  servo3.write (flex_sensor3);
  servo4.write (flex_sensor4); //:0
  servo5.write (flex_sensor5);

  Serial.println (flex_sensor1);
  Serial.println (flex_sensor2);
  Serial.println (flex_sensor3);
  Serial.println (flex_sensor4);
  Serial.println (flex_sensor5);

  delay (250);
}
```

Lecture des capteurs

La fonction
`analogRead()`
convertit la tension
en nombre entre 0 et
1023.

La valeur dépend
directement de la
flexion du doigt.

Conversion en angle

la fonction `map()`
transforme la
valeur du capteur
en angle moteur.

Exemple :
doigt droit $\rightarrow 0^\circ$
doigt plié $\rightarrow 180^\circ$

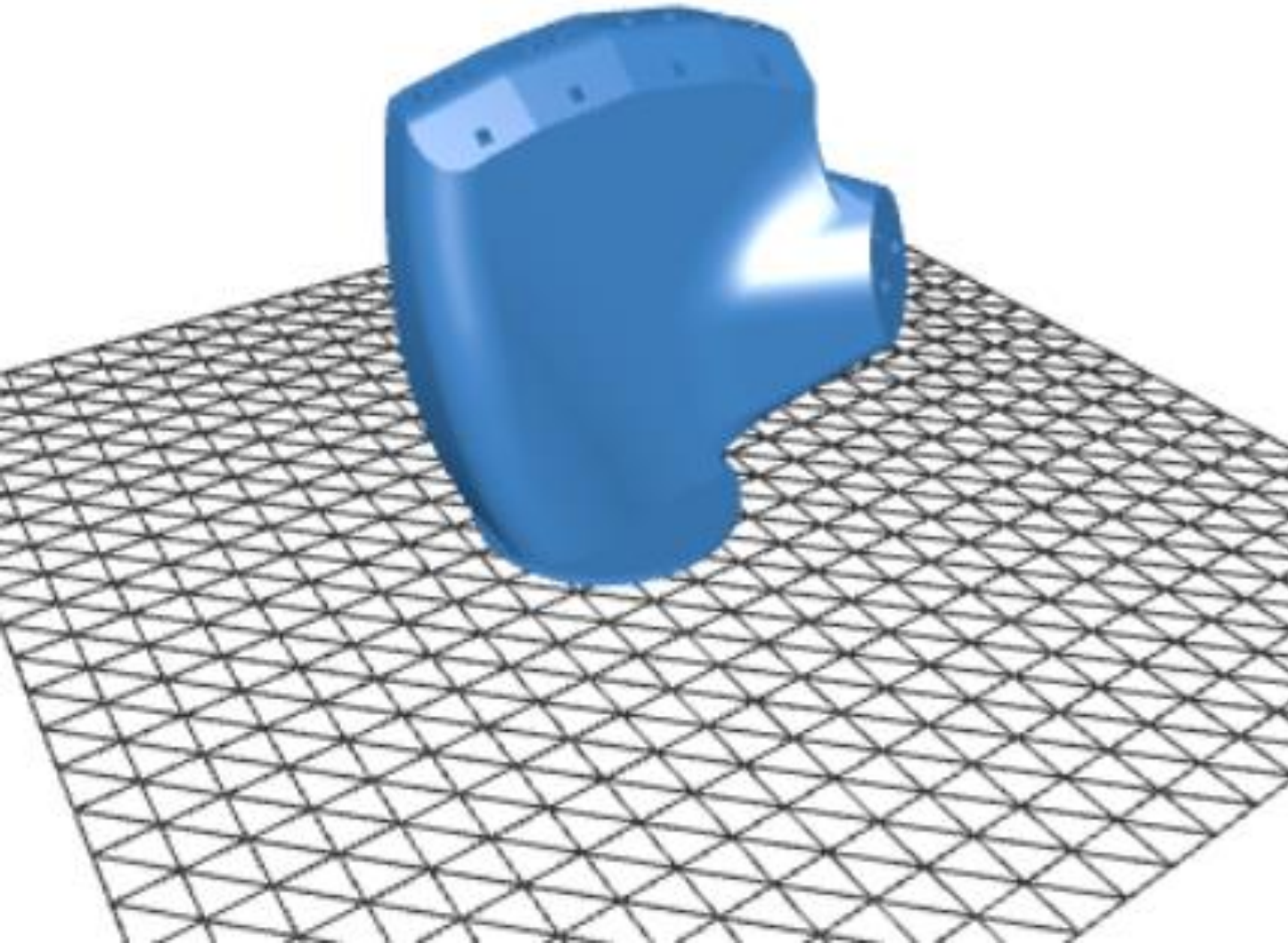
Une calibration
est nécessaire car
chaque capteur
est différent.

Mouvement du moteur

La fonction
`servo.write()`
génère un signal
PWM.

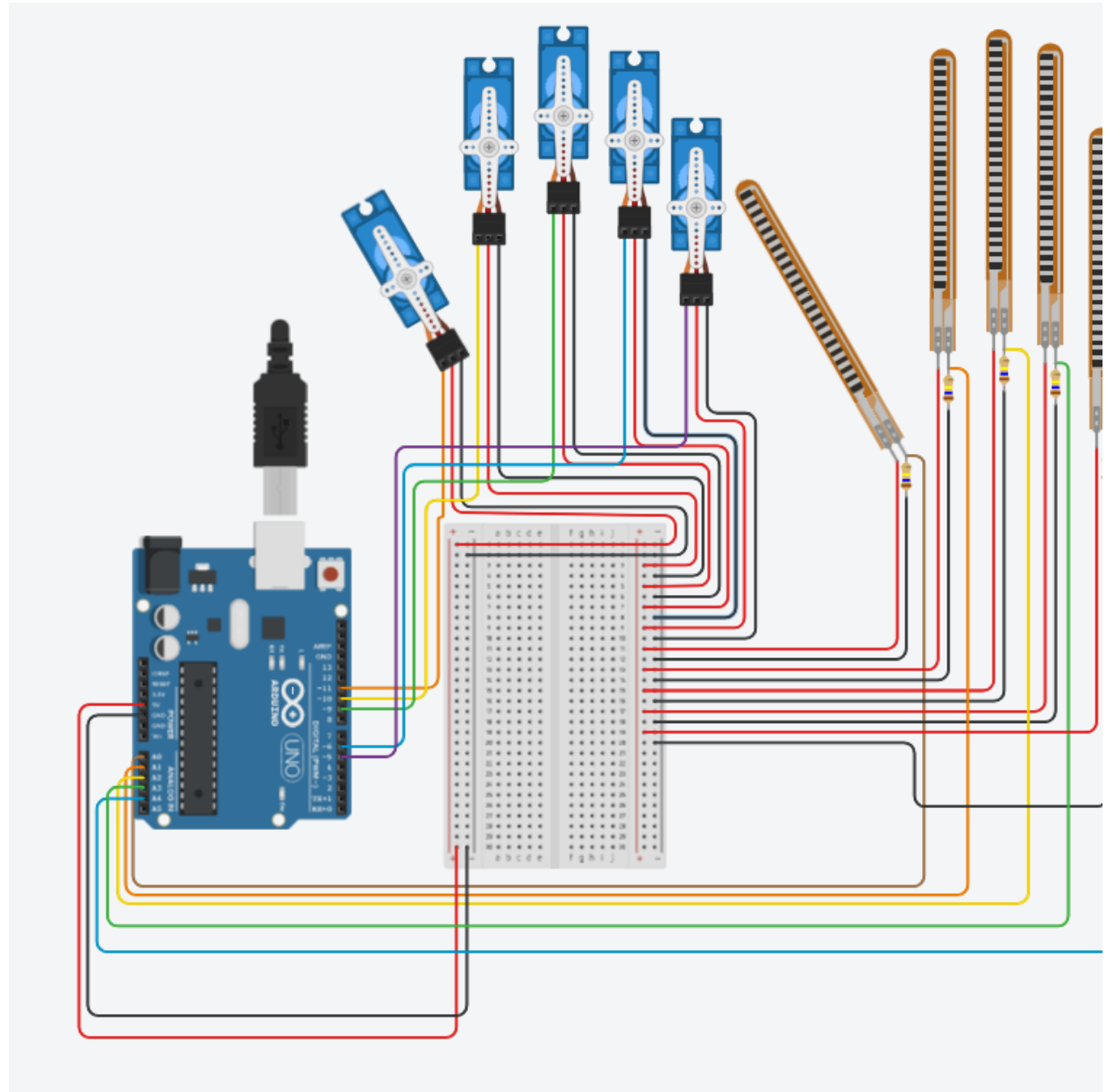
Le servo se
positionne
immédiatement à
l'angle demandé.

Modélisation 3D



- main a été conçue en modélisation 3D.
- Elle comporte :
- doigts articulés
- passages de câbles
- supports de servomoteurs
- structure légère
- L'objectif est de reproduire la biomécanique humaine.

Shemas de cablage



Reconnaissance des gestes de la main :

IA + webcam pour reconnaître les gestes de la main en temps réel.

```
1
2 # Projet : Reconnaissance des gestes de la main
3
4 # Importation des modules nécessaires
5 import cv2
6 import numpy as np
7 import mediapipe as mp
8 import tensorflow as tf
9 # From tensorflow.keras.models import load_model # ancienne méthode, non utilisée ici
10
11 # Initialisation de Mediapipe pour détecter une main
12 mpHands = mp.solutions.hands
13 hands = mpHands.Hands(max_num_hands=1, min_detection_confidence=0.7)
14 mpDraw = mp.solutions.drawing_utils
15
16 # Chargement du modèle d'IA pour reconnaître les gestes
17 model = tf.keras.models.load_model(
18     'hand_gesture_detection\mp_hand_gesture')
19
20 # Lecture des noms des gestes (classes)
21 f = open('hand_gesture_detection\gesture.names', 'r')
22 classNames = f.read().split('\n')
23 f.close()
24 print(classNames)
25
26 # Ouverture de la webcam
27 cap = cv2.VideoCapture(0)
28
29 while True:
30     # Lecture d'une image de la webcam
31     _, frame = cap.read()
32
33     x, y, c = frame.shape
34
35     # Mise en miroir de l'image (plus naturel pour l'utilisateur)
36     frame = cv2.flip(frame, 1)
37     framergb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
38
39     # Détection de la main dans l'image
40     result = hands.process(framergb)
41
42     className = ''
43
44     # Traitement de la détection si une main est trouvée
45     if result.multi_hand_landmarks:
46         landmarks = []
47         for handslms in result.multi_hand_landmarks:
48             for lm in handslms.landmark:
49                 print(id, lm)
50                 lmx = int(lm.x * x)
51                 lmy = int(lm.y * y)
52
53                 landmarks.append([lmx, lmy])
54
55     # Dessin des points et connexions de la main
56     mpDraw.draw_landmarks(frame, handslms, mpHands.HAND_CONNECTIONS)
57
58     # Prédiction du geste réalisé
59     prediction = model.predict([landmarks])
60     classID = np.argmax(prediction)
61     className = classNames[classID]
62
63     # Affichage du geste détecté sur l'écran
64     cv2.putText(frame, className, (10, 50), cv2.FONT_HERSHEY_SIMPLEX,
65                 1, (0, 0, 255), 2, cv2.LINE_AA)
66
67     # Affichage de l'image finale dans une fenêtre
68     cv2.imshow("Output", frame)
69
70     # Quitter le programme en appuyant sur 'q'
71     if cv2.waitKey(1) == ord('q'):
72         break
73
74 # Fermeture de la webcam et des fenêtres
75 cap.release()
76 cv2.destroyAllWindows()
```

Objectif :

Détecter la main, analyser les points, reconnaître le geste, afficher le résultat

Outils :

OpenCV (webcam)

MediaPipe (points de la main)

TensorFlow (IA)

Fonctionnement :

1. Capture image

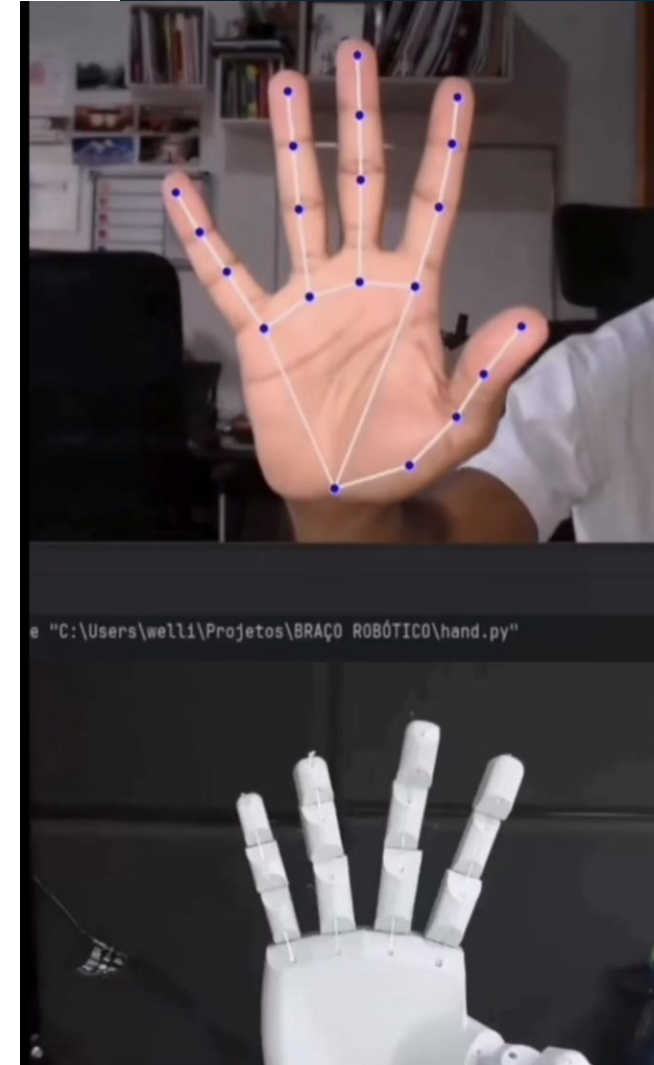
2. Détection main

3. Extraction points

4. Prédiction geste

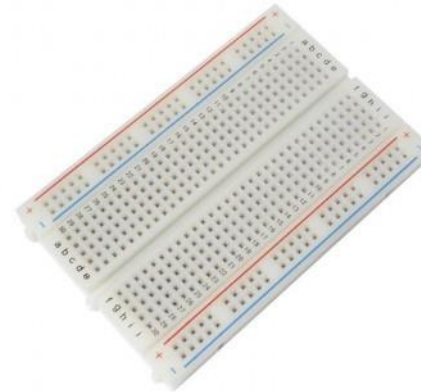
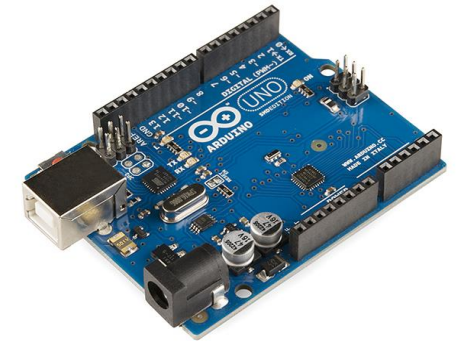
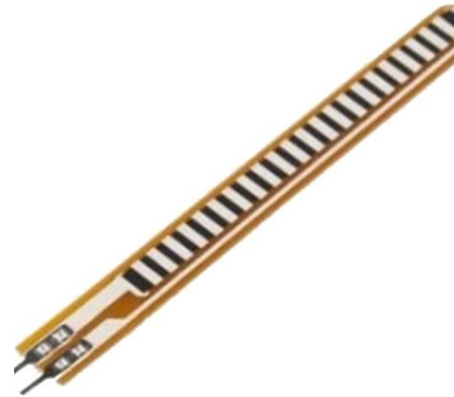
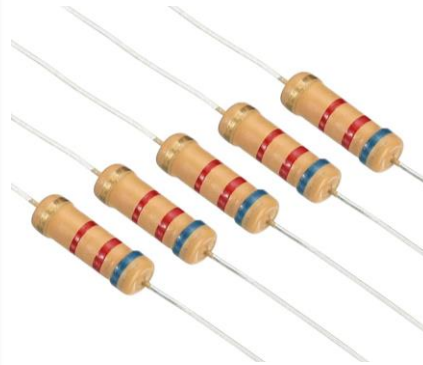
Résultat :

Affichage en direct du geste reconnu. Contrôle simple par la main.



Materiel utiliser :

- Arduino UNO
- Breadboard
- 5 capteurs flexion
- 5 résistances
- 5 servomoteurs
- Fils Dupont



Problèmes anticipés

- **Câblage** : mauvais branchement possible
- **Résistances / capteurs** : valeurs différentes → mesure imprécise
- **Codage** : erreurs de conversion angle , calibration et module